

El proyecto puede ser encontrado en github a través del siguiente enlace:

<https://github.com/lyraxihe/Grafica>

A continuación voy a explicar por apartados en orden de realización los distintos elementos que se encuentran en mi proyecto.

Mesh:

Para comenzar, se tomó como base el ejemplo “6 mesh loader”. A partir de él, se creó una clase dedicada a la gestión de shaders y otra para cada objeto, las cuales poseen sus propios vertex y fragment shaders. Esta separación permitió renderizar distintos objetos de manera sencilla dentro de la clase *Scene*.

Dentro de la clase *Object* figura la función *load_mesh*, la cual se encarga de:

- Recibir el *file path* del mesh.
- Almacenar las coordenadas UV (en caso de estar presentes) para permitir la correcta aplicación de texturas posteriormente.

Esta modularidad facilita la incorporación de nuevos modelos sin afectar al resto de la arquitectura.

Texturas:

La asignación de texturas se basó en el ejemplo “7 2d textures”, adaptándolo para que las texturas se apliquen sobre un mesh en lugar de primitivas (como un cubo). Para ello, se implementó en la clase *Object* la función *create_texture_2d*, que:

- Recibe las coordenadas UV del mesh.
- Utiliza la librería SOIL para cargar la imagen y generar la textura.
- Envía los datos a la GPU.

Durante el proceso de renderizado, la función *render* vincula la textura, enlaza el VAO y dibuja el objeto aplicándole la transformación (traslación, rotación y escalado) correspondiente.

Transparencia:

Para la correcta representación de objetos transparentes (en mi caso, un corazón), se realizó lo siguiente en la clase *Scene*:

- Antes de renderizar el objeto transparente, se deshabilita la escritura en el Z-buffer con `glDepthMask(GL_FALSE)`.
- Se habilita el blending mediante `glEnable(GL_BLEND)` y se configura la función de mezcla con `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`, lo que permite interpretar adecuadamente el canal alpha.
- Finalmente, después de renderizar el objeto transparente, se restaura la configuración original (habilitando nuevamente la escritura en el Z-buffer y deshabilitando el blending).

Esta metodología asegura que los objetos transparentes se mezclen correctamente con el fondo.

Skybox:

El skybox se implementó tomando como referencia el ejemplo “9 cubemap”. Se encapsuló el código en una clase propia que posee su propio shader, de modo que:

- El skybox administra sus coordenadas y recibe una ruta de texturas cuyo nombre se incrementa numéricamente para obtener cada una de sus caras.
- Se renderiza siguiendo la posición de la cámara, generando la sensación de un entorno infinito.

Esta técnica evita alcanzar “el final” del skybox.

Movimiento de cámara:

El sistema de cámara se adaptó del ejemplo “9 cubemap”. Se implementó una cámara interactiva que:

- Responde a clics del mouse y a las teclas *W*, *A*, *S*, *D*.
- Actualiza la matriz inversa de transformación (*transform_matrix_inverse*), la cual se utiliza como la *view_matrix* al renderizar cada objeto.

Terreno:

Para la creación del terreno se tomó como base el ejemplo “10 height map”. Se desarrolló una clase específica con un shader propio, en la que se:

- Utiliza un *texture path* para cargar un mapa de altura.
- Se genera una malla de elevación en función de dicho mapa, asignando adecuadamente las coordenadas en el eje y.
- El fragment shader aplica un gradiente de color (entre rosa y verde) según los niveles de altura.

Grafo de escena:

Para generar un grafo de escena (en mi caso con mariposas) se cumplieron los siguientes pasos:

- Se ha modificado la clase `Object` para incluir un puntero al objeto *padre*.
- Durante el proceso de renderizado, cada objeto verifica si tiene asignado un padre. En caso afirmativo, multiplica sus parámetros de traslación, rotación, ángulo y escala por la transformación acumulada del padre.
- Se implementa una función `setParent` en la clase `Scene` para definir estas relaciones jerárquicas.

Iluminación:

Se utiliza el modelo Blinn-Phong para la iluminación, agrupando los parámetros de luz en un bloque uniforme (*LightBlock*). En el vertex shader se transforman las posiciones y normales, y en el fragment shader se calculan los componentes:

- **Ambiental:** Luz base constante.
- **Difusa:** Basada en el ángulo entre la luz y la normal.
- **Especular:** Usando el half-vector (promedio entre la dirección de luz y la vista) controlado por *shininess*.

Estos componentes se combinan con el color de la textura para lograr el efecto.

Postprocesado Bloom:

Se desarrollaron shaders específicos para cada etapa:

- **Threshold Shader:** Extrae las áreas brillantes de la escena.
- **Blur Shader:** Aplica un desenfoque (mediante un bucle de ping-pong) sobre la imagen extraída.
- **Shader Final:** Combina la escena original con la imagen desenfocada para generar el halo luminoso característico del Bloom.

Además, se expusieron parámetros ajustables (como *threshold*, cantidad de iteraciones de blur y *exposure*) para permitir una optimización y personalización del efecto.